

i DAT516/DAT690/DIT516 Exam Instructions

Write your answers in Inspira. You can of course use separate sheets of paper to draft and experiment, but only the answers in Inspira will be graded. The answers can and should be short.

Python interpreters such as Self Practice are not available. Hence you have to reason about the answers on your own. Using pencil and paper is a good aid for this.

You can also have with you an A4 paper, hand-written on both sides, with whatever information you want to have available

You will need 24 points out of 60 in this exam to get accepted with grade 3, 36 points for grade 4, and 48 points for grade 5. The exam grade will also be the final grade for the course.

If used as a re-exam for DAT515 or DIT515 from earlier years, the final grade of the course will be the grade from your labs, provided that you get at least 24 points in this exam.

i Python Syntax

For your reference: the syntax of (the relevant parts of) Python is attached.

1 Question 1: types and values

Question 1 (16 p). What is the value and type of the following expressions? Remember that None is also a value! It can also happen that the expression contains an error. In that case, indicate the kind of error: one of `SyntaxError`, `TypeError`, `AttributeError`, `ZeroDivisionError`, `KeyError`, `NameError`, `IndexError`, and explain (in your own words) the reason of the error. (1+1 p for each part)

```
{4,5,6}.add(5)
```

Fill in your answer here

```
{n%3 for n in range(1, 10, 3)}
```

Fill in your answer here

```
set([1, 2, 2, 1])
```

Fill in your answer here

```
(lambda x, y: y(x, x+1))(2, max)
```

Fill in your answer here

```
set([1, 2, 2, 1])
```

Fill in your answer here

```
'a cat' is 'a cat'
```

Fill in your answer here

```
{m: m%3 in range(1,4)}
```

Fill in your answer here

```
len({print(n) for n in range(1, 100) if 10 < n < 20})
```

Fill in your answer here

Maximum marks: 16

2 Question 2: comprehensions

Question 2 (12 p). A bilingual dictionary maps the words of one language to another language. In general, the mapping is not one-to-one, because a word can have many translations. Therefore, the values in the dictionary are lists. Here is an example: a small fragment of a Swedish-English dictionary:

```
sweng = {
    'farmor': ['grandmother', 'granny'],
    'mormor': ['grandmother', 'granny'],
    'mor': ['mother', 'mum'],
    'mamma': ['mother', 'mum'],
    'bror': ['brother'],
    'broder': ['brother'],
    # etc, 100,000 words
}
```

Question 2.1, 6 points: your first task is to convert this to an English-Swedish dictionary with the same structure: the keys are English words and the values are lists of Swedish words. The translation relation must be preserved: the list must contain exactly those Swedish words that can be translated to the English words in the original dictionary. Your answer must be one expression, which refers to `sweng` but would work for any dictionary with the same structure.

Fill in your answer here

Question 2.2, 6 points: your second task is write expressions that compute statistics about the dictionary `sweng` (assuming it has been expanded to its full size, not the sample shown above). It should consist of giving values to the following variables:

```
eng_words: int # the number of distinct English words
swe_length: float # the average length of Swedish words
same_words: int # the number of Swedish words that have the same word as one of its translations
```

An example of a word that counts in the last one is "finger".

The values must be given as single expressions, not statements such as `for-loops`.

Fill in your answer here

Maximum marks: 12

3 Question 3: Inheritance

Question 3 (16 p). The following class defines dates as combinations of month and day.

```
class Date:
    def __init__(self, month, day):
        self._month = month
        self._day = day

    def __str__(self):
        return f'{self._day}/{self._month}'

    def __lt__(self, other):
        return (self._month, self._day) < (other._month, other._day)
```

Notice, in particular, the dunder method `__lt__`, which defines the ordering relation of times. It is defined by comparing two tuples, where the first components are compared first; if they are different, the comparison is decided. If they equal, the second components are compared.

Question 3.1 (4p). Given the two dates:

```
date1 = Date(4, 20)
date2 = Date(10, 14)
```

What is printed by the following?

```
print(date1, '<', date2, date1 < date2)
```

Fill in your answer here

Question 3.2 (6p). Define a similar class `Time`, which has the variables `_hour` and `_minute` and the dunder methods `__str__()` and `__lt__()`. The `__str__()` method should produce strings such as `'23:45'`. The `__init__()` method should raise `ValueError` if any of the given arguments is not in the correct ranges: hours from 0 to 23 and minutes from 0 to 59. (An analogous check should have been performed in the `Date` class as well, but was omitted because it would require more space.)

Fill in your answer here

Question 3.3 (6p) The final task is to define the class `DateTime`, which inherits from both `Date` and `Time`. Its `__init__()` method is a bit tricky, because it involves both classes separately. Therefore we show it here, together with headers of two methods that extract a `Date` and a `Time` from a `DateTime`.

```
class DateTime(Date, Time):
    def __init__(self, month, day, hour, minute):
        Date.__init__(self, month, day)
        Time.__init__(self, hour, minute)

    def date(self):
        # TODO return a Date

    def time(self):
        # TODO return a Time
```

In addition to implementing the `date()` and `time()` methods, you should also define the dunder methods `__str__()` and `__lt__()`. Do this with a maximal reuse of inherited methods and the new methods `date()` and `time()`; you will lose points if you write more code than necessary. Make sure that the methods produce the output shown in the following:

```
>>> datetime1 = DateTime(4, 20, 22, 35)
>>> datetime2 = DateTime(10, 14, 13, 45)
>>> print(datetime1, '<', datetime2, datetime1 < datetime2)
20/4 at 22:35 < 14/10 at 13:45 True
```

Fill in your answer here

Maximum marks: 16

4 Question 4: HTML API

Question 4 (16 p)

Attached is the API for a (fictitious) Python library for parsing and generating HTML. All answers in this section should make use of this library, respecting the documented types. Your code does not need to include type hints.

4.1 (4 p)

Construct a `Document` object representing the HTML document below, storing it in a variable `doc`. You can ignore newlines and indentation.

```
<!DOCTYPE html>
<html lang="en">
  <body>
    <p>This text is <strong>bold</strong></p>
    <p role="footer">Here is more text</p>
  </body>
</html>
```

Fill in your answer here

4.2 (4 p)

Define a function `count_paras` which returns the number of `<p>` tags in a `Document`. For the example above it should return 2:

```
>>> count_paras(doc)
2
```

Fill in your answer here

4.3 (6 p)

Implement the `__str__(self)` method of both `Elem` and `Document` classes, such that calling `str(doc)` will return the HTML document as a string, as in the example below. You can ignore newlines and indentation.

```
>>> str(doc)
'<!DOCTYPE html>\n<html lang="en"><body><p>This text is <strong>bold</strong></p><p role="footer">Here is more t
```

Fill in your answer here

4.4 (2 p)

Finally, we want to test the correctness of the implementation of your `Elem.__str__` method. Write some code which asserts that calling `str()` on a object of class `Elem` and then parsing the resulting string gives you back an `Elem` equal to the original one.

Fill in your answer here

Maximum marks: 16

Document 2

Attached



```

<stm> ::= <decorator>* class <name> (<name>,*):? <block>
      | <decorator>* def <name> (<arg>,*): <block>
      | import <name> <asname>?
      | from <name> import <imports>
      | <exp>,* = <exp>,*
      | <exp> <assignop> <exp>
      | for <name> in <exp>: <block>
      | <exp>
      | return <exp>,*
      | yield <exp>,*
      | if <exp>: <block> <elses>?
      | while <exp>: <block>
      | pass
      | break
      | continue
      | try: <block> <except>* <elses> <finally>?
      | assert <exp> ,<exp>?
      | raise <name>
      | with <exp> as <name>: <block>

<decorator> ::= @ <exp>
<asname>    ::= as <name>
<imports>   ::= * | <name>,*
<elses>     ::= <elif>* else: <block>
<elif>      ::= elif <exp>: <block>
<except>    ::= except <name>: <block>
<finally>   ::= finally: <block>
<block>     ::= <stm> <stm>*
<exp>       ::= <exp> <op> <exp>
      | <name>?.?<name>(<arg>,* )
      | <literal>
      | <name>
      | ( <exp>,* )
      | [ <exp>,* ]
      | { <exp>,* }
      | <exp>[<exp>]
      | <exp>[<slice>,*]
      | lambda <name>*: <exp>
      | { <keyvalue>,* }
      | ( <exp> for <name> in <exp> <cond>? )
      | [ <exp> for <name> in <exp> <cond>? ]
      | { <exp> for <name> in <exp> <cond>? }
      | { <exp>: <exp> for <name> in <exp> <cond>? }
      | - <exp>
      | ~ <exp>
      | not <exp>
      | <name> := <exp>

<keyvalue>  ::= <exp>: <exp>
<arg>       ::= <name>
      | <name> = <exp>
      | *<name>
      | **<name>

<cond>      ::= if <exp>
<op>        ::= + | - | * | ** | / | // | % | @ | == | > | >= | < | <= | !=
      | in | not in | and | or | & | | | ^ | << | >> | :=
<assignop>  ::= += | -= | *= | /= | %= | &= | |= | ^=
<slice>     ::= <exp>? :<exp>? <step>?
<step>      ::= :<exp>?

```

Question 4

Attached



Type annotation syntax

Expression	Explanation
<code>x: T</code>	Variable <code>x</code> has type <code>T</code>
<code>def f (x: T) -> U</code>	Function <code>f</code> takes a parameter <code>x</code> of type <code>T</code> and returns a value of type <code>U</code>
<code>T U</code>	Either type <code>T</code> or type <code>U</code>
<code>list[T]</code>	A list of items of type <code>T</code>
<code>dict[T, U]</code>	A dictionary with keys of type <code>T</code> and values of type <code>U</code>
<code>Iterator[T]</code>	An iterator over items of type <code>T</code>

HTML Library API

```
# An HTML element
class Elem:
    # Name of the tag of this element
    tag: str

    # Attributes of this element
    attrs: dict[str, str]

    # Child elements of this element can be sub-elements or text
    elems: list[Elem | str]

    # Initialisation method
    def __init__(
        self,
        tag: str,
        attrs: dict[str, str] = {},
        elems: list[Elem | str] = []
    )

    # Equality check method
    # Returns True if `other` is an Elem which is equal to `self`
    # Recurses through all child elements
    def __eq__(self, other) -> bool

    # Add a child element
    def add_elem(self, elem: Elem) -> None

# A complete HTML document
class Document:
    # Optional text that comes before the root element
    prolog: str | None

    # The root element of the document tree
    root: Elem

    # Equality check method
    # Returns True if `other` is a Document which is equal to `self`
    # Recursively checks entire document tree
    def __eq__(self, other) -> bool

    # Return an iterator over all elements with the given tag name
    # Recurses through entire document tree
    def iter_tag(self, tag: str) -> Iterator[Elem]
```

```
# Return an iterator over all elements with the given attribute
# Recurses through entire document tree
def iter_attr(self, key: str, value: str) -> Iterator[Elem]

# Return an iterator over all text content in the document
# Recurses through entire document tree
def iter_text(self) -> Iterator[str]

# Read a file and parse it as HTML
# Raises IOError if file cannot be read
# Raises ParseError if file contents cannot be parsed as HTML
def parse_file(filepath: str) -> Document

# Parse a string as HTML
# Raises ParseError if string cannot be parsed as HTML
def parse_string(raw: str) -> Elem
```